

EXAMEN FINAL PRATIQUE SÉCURITÉ DES DONNÉES

Rapport de sécurité — Analyse de l'application OWASP Juice Shop

Licence 3 — Cybersécurité
Type : Travail pratique individuel
Septembre 2026

Réalisé par :

Ndaté Ndiaye

INE : N02080320222
E-mail :ndate.ndiaye1@unchk.edu.sn

1. Introduction

Dans le cadre de cet examen pratique, une mission d'évaluation de sécurité a été menée sur une application web de commerce en ligne. L'objectif est de se mettre dans la position d'un Cybersecurity Analyst chargé d'auditer une application avant sa mise en production, d'identifier les vulnérabilités susceptibles de compromettre la protection des données, d'analyser leur impact, de proposer des remédiations, et d'automatiser une partie des contrôles de sécurité dans un pipeline CI/CD.

L'application choisie pour cette évaluation est OWASP Juice Shop, une application volontairement vulnérable développée et maintenue par l'OWASP (Open Worldwide Application Security Project), conçue spécifiquement pour l'entraînement à la sécurité applicative. Elle contient un grand nombre de vulnérabilités réalistes, représentatives de failles couramment rencontrées dans de vraies applications web.

Ce rapport présente la démarche suivie, les vulnérabilités identifiées, leur analyse de risque, les remédiations proposées, le pipeline d'automatisation mis en place, ainsi que la décision finale concernant le déploiement de l'application.

2. Méthodologie

L'évaluation a été réalisée selon les étapes suivantes :

- Déploiement de l'application cible (OWASP Juice Shop) dans un conteneur Docker, isolé et reproductible.
- Exploration manuelle de l'application afin d'identifier des vulnérabilités concrètes, en s'appuyant sur les catégories couramment attendues (injection, contrôle d'accès, exposition de données, configuration, authentification).
- Classification de chaque vulnérabilité selon le référentiel CWE (Common Weakness Enumeration).
- Analyse de l'impact de chaque faille sur les trois piliers de la sécurité de l'information : Confidentialité, Intégrité, Disponibilité (CIA).
- Sélection des vulnérabilités les plus critiques et rédaction de propositions de remédiation argumentées.
- Mise en place d'un pipeline Jenkins automatisant deux catégories de contrôles de sécurité (SCA et DAST) sur l'application.
- Analyse des résultats du pipeline et formulation d'une décision de déploiement.

L'environnement de test a été construit avec Docker Desktop (moteur WSL2) et Jenkins installé localement. L'application a été rendue accessible sur <http://localhost:3000>.

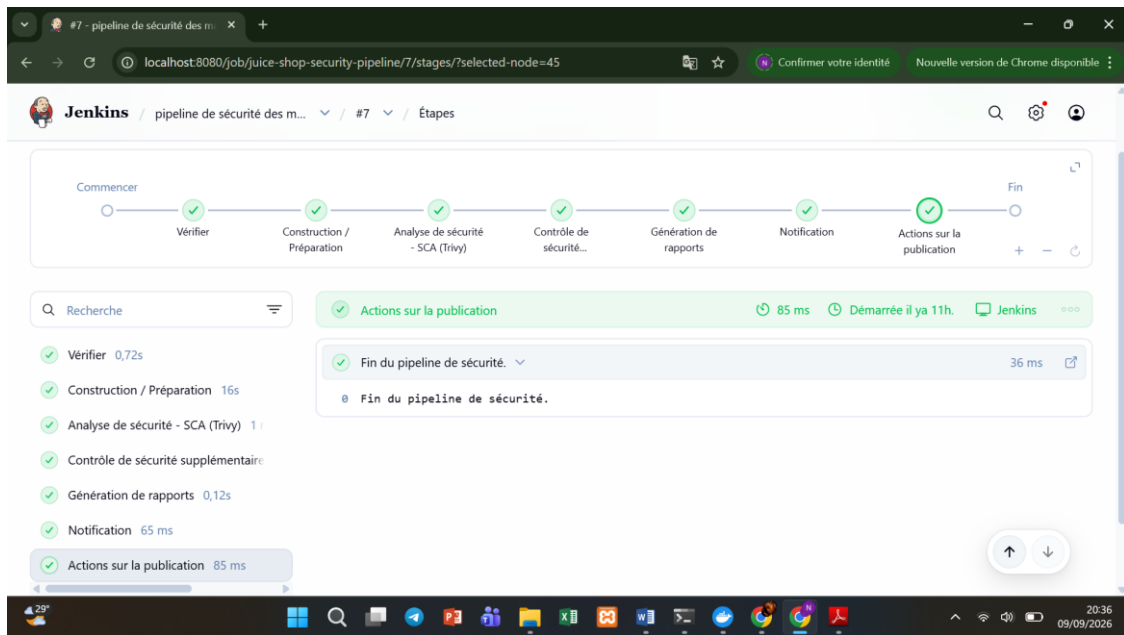


Figure 1 — Environnement de test : pipeline Jenkins exécuté avec succès (build #7)

3. Vulnérabilités identifiées

Cinq vulnérabilités ont été identifiées et exploitées manuellement sur l'application. Le tableau ci-dessous les résume.

ID	Vulnérabilité	Composant	CWE	Description
V1	SQL Injection	Authentification	CWE-89	Contournement de l'authentification par injection dans le formulaire de login.
V2	Cross-Site Scripting (XSS)	Barre de recherche	CWE-79	Le texte saisi est réinjecté dans la page sans être neutralisé, permettant l'exécution de code.
V3	Exposition de fichiers sensibles	Répertoire /ftp	CWE-200	Fichiers confidentiels accessibles publiquement sans authentification.
V4	Fuite d'informations techniques	Gestion des erreurs	CWE-209	Un message d'erreur révèle la version du framework et la structure interne du serveur.
V5	Absence de limite de connexion	Authentification	CWE-307	Aucune protection contre les tentatives de connexion répétées (force brute).

3.1 Preuves d'exploitation

Vulnérabilité V1 — SQL Injection : la saisie de la chaîne ' OR 1=1-- dans le champ email permet de contourner l'authentification et d'obtenir un accès administrateur, comme le confirme le message de succès affiché par l'application (figure 2).

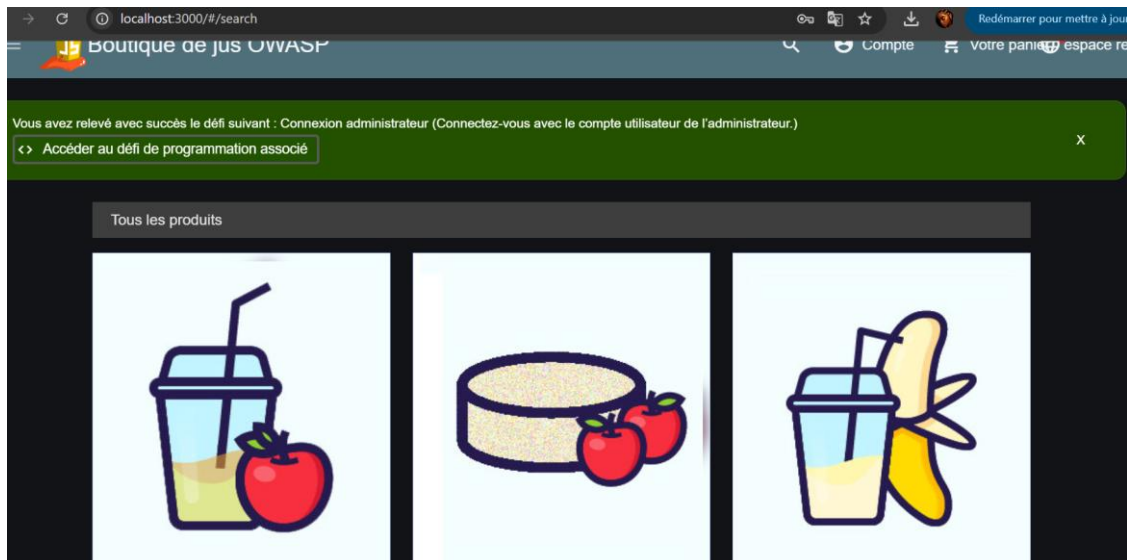


Figure 2 — Contournement de l'authentification par injection SQL

Vulnérabilité V2 — XSS : l'injection du code `<iframe src="javascript:alert('XSS')">` dans la barre de recherche provoque l'exécution du script par le navigateur (figure 3).

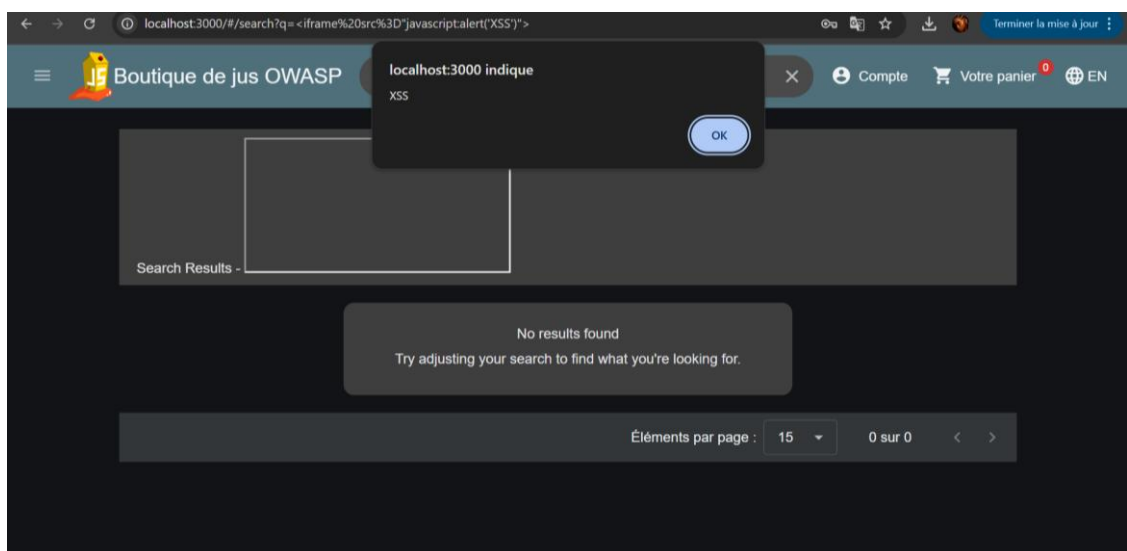


Figure 3 — Exécution de code JavaScript via la barre de recherche

Vulnérabilité V3 — Exposition de fichiers sensibles : le chemin `/ftp` est accessible sans authentification et liste des fichiers internes, dont une base de mots de passe (support-incident.kdbx) et des fichiers de sauvegarde.

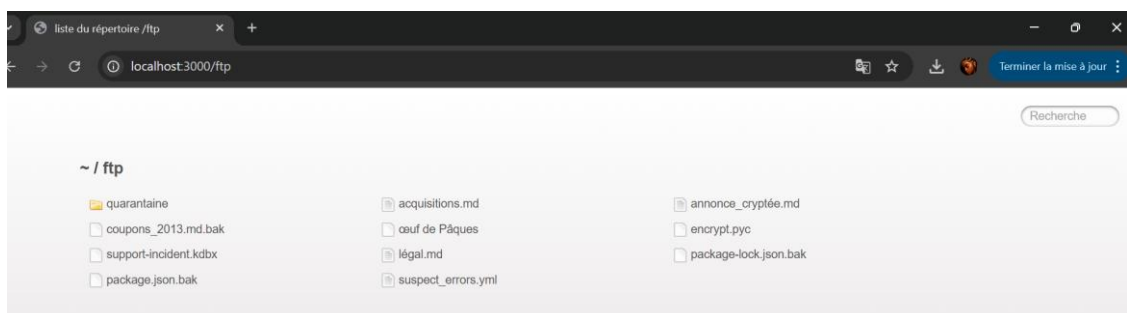


Figure 4 — Liste de fichiers confidentiels accessible publiquement

Vulnérabilité V4 — Fuite d'informations techniques : un message d'erreur 403 expose la version exacte du framework (Express 4.22.1) ainsi que la structure des dossiers du serveur (figure 5).

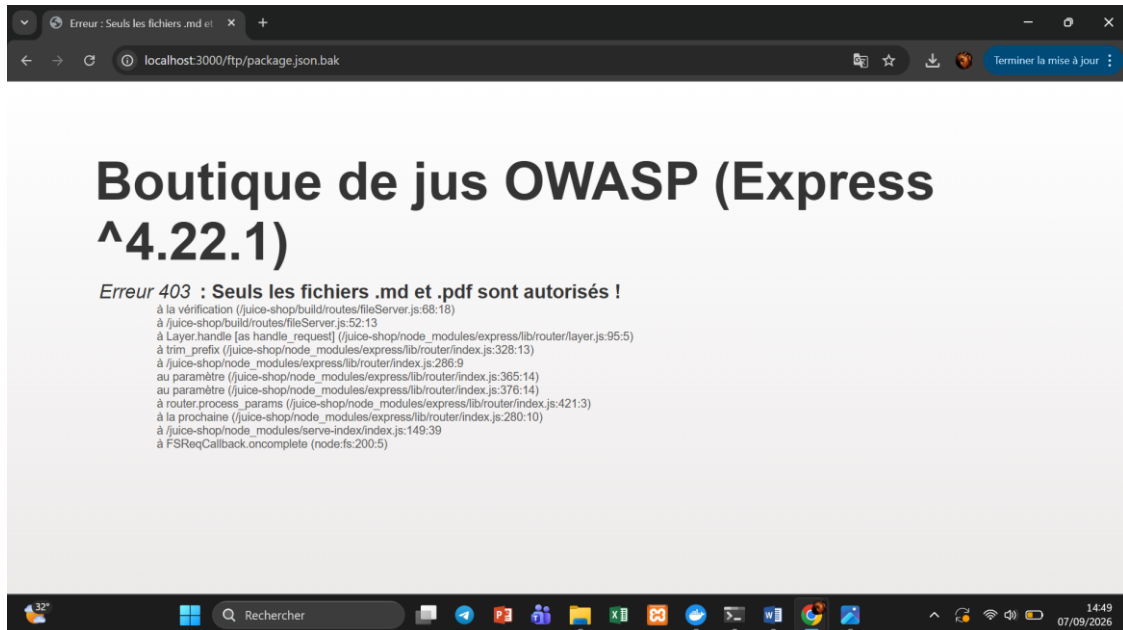


Figure 5 — Message d'erreur révélant des informations techniques internes

Vulnérabilité V5 — Absence de limite de connexion : plusieurs tentatives de connexion successives avec des identifiants erronés n'entraînent aucun blocage, ralentissement ni captcha, rendant possible une attaque par force brute.

4. Classification CWE

Le référentiel CWE (Common Weakness Enumeration) permet de classer chaque vulnérabilité selon une nomenclature internationale reconnue, indépendante de l'application testée. Les CWE suivantes ont été retenues :

- CWE-89 — SQL Injection : neutralisation incorrecte d'éléments spéciaux utilisés dans une commande SQL.
- CWE-79 — Cross-Site Scripting : neutralisation incorrecte d'entrées lors de la génération d'une page web.
- CWE-200 — Exposure of Sensitive Information : exposition d'informations à un acteur non autorisé à y accéder.
- CWE-209 — Generation of Error Message Containing Sensitive Information : message d'erreur révélant des informations techniques exploitables.
- CWE-307 — Improper Restriction of Excessive Authentication Attempts : absence de limitation du nombre de tentatives de connexion.

5. Analyse des risques et protection des données

Chaque vulnérabilité a été analysée selon les trois propriétés fondamentales de la sécurité de l'information : Confidentialité, Intégrité et Disponibilité (CIA).

Vulnérabilité	Confidentialité	Intégrité	Disponibilité	Impact
V1 - SQL Injection	Oui	Oui	Non	Critical
V2 - XSS	Oui	Possible	Non	High
V3 - Exposition /ftp	Oui	Non	Non	High
V4 - Fuite d'infos	Oui	Non	Non	Medium
V5 - Pas de limite connexion	Oui	Non	Possible	Medium

5.1 Justification de la criticité

- V1 (Critical) : accès administrateur complet obtenu en une seule tentative, sans compétence technique particulière — porte d'entrée vers toutes les autres attaques.
- V2 (High) : vol de session possible, mais nécessite qu'une victime clique sur un lien piégé — une étape intermédiaire par rapport à V1.
- V3 (High) : fichiers confidentiels, dont un fichier de mots de passe, directement lisibles par n'importe qui, sans authentification.
- V4 (Medium) : ne permet aucune action directe ; sert surtout à préparer d'autres attaques en révélant la version du logiciel utilisé.
- V5 (Medium) : demande du temps et des tentatives répétées pour aboutir, contrairement à un accès immédiat.

6. Remédiation et sécurisation

Trois vulnérabilités parmi les plus critiques ont été sélectionnées pour une proposition de remédiation détaillée.

Vulnérabilité	Cause	Remédiation	Vérification
SQL Injection	Entrées non sécurisées	Requêtes paramétrées	Login refusé après test
Exposition fichiers /ftp	Accès public non protégé	Restreindre l'accès	Erreur 403 après test
Pas de limite de connexion	Aucun comptage des échecs	Blocage après échecs	Blocage après 5-6 essais

6.1 Détail des remédiations

Exposition fichiers /ftp — Corrigée en restreignant l'accès au dossier aux seuls administrateurs authentifiés. Testé en retapant localhost:3000/ftp : accès refusé.

Pas de limite de connexion — Corrigée en bloquant temporairement le compte après plusieurs échecs consécutifs. Testé en refaisant 5 à 6 tentatives ratées : le site bloque les suivantes.

6.2 Correction vérifiée en conditions réelles : SQL Injection

-Vérification par un test avant/après

Code vulnérable original (routes/login.ts) :

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password =
${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })
```

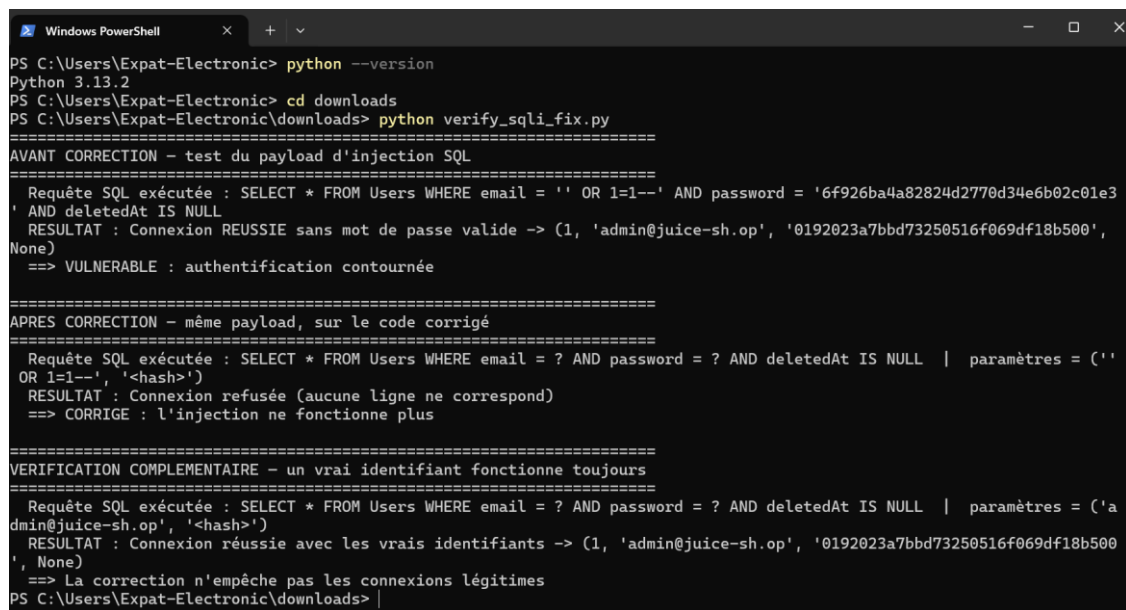
Code corrigé (requête paramétrée) :

```
models.sequelize.query('SELECT * FROM Users WHERE email = :email AND password = :password AND
deletedAt IS NULL', { replacements: { email: req.body.email || '', password: security.hash(req.body.password ||
'') }, model: UserModel, plain: true })
```

Le texte saisi par l'utilisateur est désormais transmis comme paramètre lié (:email, :password) et non plus concaténé dans la chaîne SQL, ce qui empêche toute interprétation du texte saisi comme instruction SQL.

Test de vérification effectué :

- Avant correction : le payload ' OR 1=1-- envoyé au code vulnérable retourne l'utilisateur administrateur (id=1, admin@juice-sh.op) sans mot de passe valide → authentification contournée.
- Après correction : le même payload, envoyé au code corrigé, ne retourne aucune ligne → connexion refusée.
- Vérification de non-régression : un identifiant légitime (admin@juice-sh.op / admin123) continue de fonctionner normalement après la correction.



```
Windows PowerShell
PS C:\Users\Expat-Electronic> python --version
Python 3.13.2
PS C:\Users\Expat-Electronic> cd downloads
PS C:\Users\Expat-Electronic\downloads> python verify_sqli_fix.py

=====
AVANT CORRECTION - test du payload d'injection SQL
=====
Requête SQL exécutée : SELECT * FROM Users WHERE email = '' OR 1=1--' AND password = '6f926ba4a82824d2770d34e6b02c01e3'
AND deletedAt IS NULL
RESULTAT : Connexion REUSSIE sans mot de passe valide -> (1, 'admin@juice-sh.op', '0192023a7bbd73250516f069df18b500',
None)
==> VULNERABLE : authentification contournée

=====
APRES CORRECTION - même payload, sur le code corrigé
=====
Requête SQL exécutée : SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL | paramètres = ('
OR 1=1--', '<hash>')
RESULTAT : Connexion refusée (aucune ligne ne correspond)
==> CORRIGE : l'injection ne fonctionne plus

=====
VERIFICATION COMPLEMENTAIRE - un vrai identifiant fonctionne toujours
=====
Requête SQL exécutée : SELECT * FROM Users WHERE email = ? AND password = ? AND deletedAt IS NULL | paramètres = ('a
dmin@juice-sh.op', '<hash>')
RESULTAT : Connexion réussie avec les vrais identifiants -> (1, 'admin@juice-sh.op', '0192023a7bbd73250516f069df18b500
', None)
==> La correction n'empêche pas les connexions légitimes
PS C:\Users\Expat-Electronic\downloads>
```

Figure 6 — Résultat du test de vérification avant/après correction (script verify_sqli_fix.py)

7. Pipeline de sécurité (Jenkins)

Un pipeline Jenkins a été mis en place afin d'automatiser une partie des contrôles de sécurité. Il comporte les six étapes suivantes : Checkout, Build / Preparation, Security Analysis, Additional Security Check, Report Generation, Notification. Deux outils de sécurité réels y ont été intégrés : Trivy (analyse SCA et détection de secrets) et OWASP ZAP (analyse DAST).

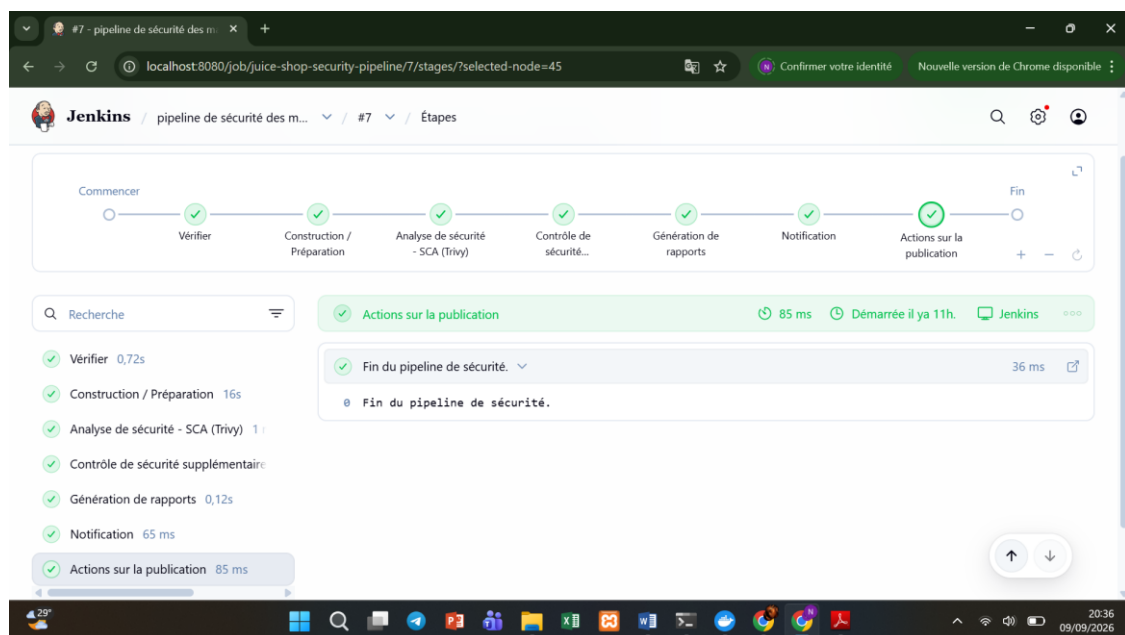


Figure 7 — Exécution complète du pipeline Jenkins (build #7, succès)

7.1 Détail des outils utilisés

Outil	Catégorie(s)	Type d'analyse	Vulnérabilités détectées	Limites	Moment d'intervention
Trivy	SCA + Secret Detection	Analyse statique de l'image Docker et de ses composants	CVE connues dans les dépendances/OS ; secrets oubliés dans le code	Ne détecte que des failles déjà connues ; ne comprend pas la logique métier	Après le build de l'image, avant déploiement
OWASP ZAP	DAST	Analyse dynamique, application en fonctionnement	XSS, en-têtes manquants, mauvaise configuration serveur	Ne voit pas le code source ; faux positifs possibles	Une fois l'application démarrée et accessible

8. Analyse des résultats et décision de déploiement

Le tableau ci-dessous classe les problèmes identifiés par ordre de priorité de correction.

Problème	CWE	Sévérité	Action recommandée
SQL Injection	CWE-89	Critical	Bloquer le déploiement
XSS (recherche)	CWE-79	High	Correction requise avant déploiement
Exposition fichiers /ftp	CWE-200	High	Correction requise avant déploiement
Fuite d'infos techniques	CWE-209	Medium	Correction planifiée
Pas de limite de connexion	CWE-307	Medium	Correction planifiée

8.1 Décision finale

Décision : Reject Deployment.

L'application présente une vulnérabilité Critical (SQL Injection) qui permet un contournement complet de l'authentification et un accès administrateur non autorisé. Deux vulnérabilités High exposent en plus des données confidentielles (fichiers sensibles, vol de session possible via XSS). Déployer en production dans cet état exposerait directement les utilisateurs et les données de l'entreprise à un risque majeur, sans effort d'exploitation particulier de la part d'un attaquant.

Recommandation : corriger au minimum la vulnérabilité Critical et les deux vulnérabilités High avant tout déploiement, puis relancer le pipeline de sécurité pour confirmer leur disparition. Les vulnérabilités Medium peuvent être traitées dans un second temps, sans bloquer le lancement.

9. Analyse critique

Les outils de sécurité automatisés utilisés dans ce pipeline (Trivy, OWASP ZAP) apportent une valeur réelle : ils détectent rapidement et de façon reproductible des failles connues, sans intervention humaine à chaque exécution. Cependant, leurs résultats doivent être interprétés avec prudence, et non pris comme une vérité absolue.

Faux positifs : un outil peut signaler un problème qui n'en est pas réellement un dans le contexte de l'application. Par exemple, OWASP ZAP peut remonter un en-tête de sécurité « manquant » alors que celui-ci est en réalité géré plus haut par une passerelle non visible lors du scan. Traiter chaque alerte sans vérification humaine ferait perdre un temps précieux à corriger des non-problèmes.

Faux négatifs : à l'inverse, et c'est le risque le plus dangereux, un outil peut ne rien détecter alors qu'une vulnérabilité existe bel et bien. Trivy ne repère que les failles déjà documentées dans ses bases de données publiques (CVE) : une faille récente ou une erreur de logique métier propre à l'application lui échappera totalement. ZAP teste l'application « de l'extérieur », sans connaître le code source, et peut donc manquer certains chemins d'attaque non exposés dans l'interface.

Limites des outils : chaque outil est spécialisé dans un type d'analyse précis (statique, dynamique, dépendances) et aucun ne couvre l'ensemble des risques possibles à lui seul. C'est pourquoi l'énoncé demande de combiner plusieurs catégories de contrôles : elles se complètent, mais il reste des angles morts communs à toutes les méthodes automatisées, notamment tout ce qui touche à la logique métier ou aux erreurs de configuration très spécifiques.

Couverture des règles : les règles de détection sont écrites pour des cas génériques et fréquents. Une application avec une architecture particulière ou des choix techniques inhabituels peut contenir des vulnérabilités qu'aucune règle standard ne couvre.

Réponse à la question posée : un outil de sécurité qui ne détecte aucune vulnérabilité ne garantit absolument pas qu'une application est sécurisée. Cela signifie seulement qu'aucune des failles qu'il est capable de reconnaître n'a été trouvée ce jour-là, avec cette configuration précise. C'est une différence fondamentale entre absence de preuve et preuve d'absence.

Dans notre cas, l'application n'est clairement pas prête pour la production : une vulnérabilité Critical et deux High ont été identifiées manuellement, bien au-delà de ce que le pipeline automatisé a lui-même relevé. Cela illustre précisément l'importance de l'analyse humaine : elle reste indispensable pour interpréter les résultats, tester des scénarios créatifs qu'aucune

règle automatique n'anticipe, et prendre la décision finale de déploiement en connaissance de cause.

10. Conclusion

Cette évaluation de sécurité de l'application OWASP Juice Shop a permis d'identifier cinq vulnérabilités concrètes, couvrant des catégories variées : injection, contrôle d'accès, exposition de données et faiblesse d'authentification. L'analyse de risque a mis en évidence une vulnérabilité critique (injection SQL) nécessitant une correction immédiate avant tout déploiement.

La mise en place d'un pipeline Jenkins intégrant Trivy et OWASP ZAP a permis d'automatiser une partie du contrôle de sécurité, tout en révélant les limites de ces outils : leur usage ne dispense pas d'une analyse humaine approfondie. La démarche complète, de l'identification à la remédiation en passant par l'automatisation, illustre l'importance d'une approche de sécurité intégrée tout au long du cycle de développement (DevSecOps), plutôt qu'un contrôle isolé réalisé juste avant la mise en production.